Certificate Chains: Compositional Proof-Carrying Code Architecture

Halley Young
Microsoft Research
halleyyoung@microsoft.com

March 23, 2026

Abstract

We present a *certificate chain* architecture for compositional proof-carrying code (PCC) in the JuGeo system. Classical PCC embeds a single monolithic proof alongside each binary; scaling this to large, modular codebases requires *compositionality*: local verifications must be independently certifiable and later assembled into a global guarantee without re-verification. We introduce ten cooperating components—`Certificate`, `CertificateChain`, `CertificateBuilder`, `CertificateStore`, `CertificateAuthority`, `CertificateVerifier`, `CertificateMerger`, `CertificateProjection`, `CertificateDiagnostics`, and `CertificateSerializer`—forming a complete lifecycle for proof certificates from issuance through chain assembly, verification, projection, and serialization. The central theorem (*Chain Integrity*) states that a chain accepted by `CertificateVerifier` constitutes a valid global verification, and that no forged chain passes verification without a valid authority signature: the trust floor of a valid chain lower-bounds the trust of every individual link. We formalize this in Lean 4 and validate it experimentally on 300 judgment instances drawn from the JuGeo benchmark suite, reporting sub-millisecond mean verification latency with zero false acceptances of forged chains.

1 Introduction

Proof-carrying code (PCC), introduced by Necula and Lee [NL97] [Nec97], attaches a machine-checkable proof to a binary, allowing a receiver to verify safety properties without trusting the producer. Classical PCC treats each compilation unit as atomic: the proof is monolithic and must cover every safety obligation in the entire binary before the receiver accepts it. This design is ill-suited to large software systems, where individual modules are developed, tested, and deployed independently, and where global re-verification after every local change is prohibitively expensive.

The JuGeo framework addresses this scalability gap through its *judgment geometry*: each coordinate in the codebase carries an 8-tuple judgment $(c, \varphi, \mathcal{A}, \mathcal{E}, \mathcal{O}, \mathcal{B}, \mathcal{T}, \Pi)$ whose evidence bundle $\mathcal{E}$ and trust annotation $\mathcal{T}$ can be checked locally. What has been missing is a *certificate layer*: a protocol by which local verifications are recorded, signed, chained together, and shipped alongside code in a form that downstream consumers can efficiently verify.

This paper presents that layer. Our main contributions are:

1. **A certificate data model** (§2): a frozen, content-addressed record carrying a coordinate, verified propositions, trust level, residual obligations, obstructions, issuer identity, and a SHA-256 signature hash.
2. **Chain construction and the weakest-link trust semantics** (§3): `CertificateChain` assembles an ordered sequence of certificates; the chain's aggregate trust is the minimum trust of any link, ensuring conservative global guarantees.
3. **A delegated authority model** (§4): `CertificateAuthority` governs issuance, revocation, and renewal within a configurable delegation depth, preventing unauthorized parties from injecting certificates into a chain.
4. **A chain-verification algorithm with the no-silent-strengthening invariant** (§5): `CertificateVerifie` checks every link for structural consistency, trust monotonicity, and the absence of inflated claims.
5. **Merger and projection** (§6): `CertificateMerger` reconciles overlapping certificates under three strategies; `CertificateProjection` produces role-appropriate views while preserving faithfulness.
6. **Serialization** (§7): `CertificateSerializer` provides full round-trip JSON fidelity for shipping certificates with compiled artifacts.
7. **Chain Integrity theorem** (§8): formally proved in Lean 4, establishing soundness and anti-forgery guarantees.

Related work. Typed assembly language (TAL) [MCG99] and proof-carrying authentication [HAT04] share our goal of shipping machine-checkable guarantees with code; unlike those systems, JuGeo operates at the level of rich judgment geometry rather than type systems alone. X.509 certificate chains [RFC5280] inspire our authority model, but our chains attest to semantic verification properties, not merely identity. Foundational PCC [App01] and LCC [Mil07] pursue similar compositionality goals; our contribution is a Python-native, geometry-aware realization integrated with the full JuGeo evidence pipeline.

2 Certificate Structure

2.1 The `Certificate` Record

A *certificate* is a signed, content-addressed attestation that a specific set of propositions has been verified at a given coordinate.

Definition 2.1 (Certificate)**.** A *certificate* is an immutable tuple

$$\mathsf{Cert} = (id,\ coord,\ props,\ tl,\ iss,\ sig,\ res,\ obs,\ exp)$$

where:

- $id \in \mathsf{UUID}$: a unique identifier.
- $coord \in \mathsf{Coordinate}$: the codebase location being attested.
- $props \subseteq \mathsf{Prop}$: the finite set of verified propositions.
- $tl \in \mathbf{TL}$: the trust level ($\mathsf{PENDING} \leq \mathsf{SETTLED} \leq \cdots$).
- $iss \in \mathcal{A}uth$: the issuing authority's name.
- $sig = \mathcal{H}(coord \| props \| tl \| iss)$: a SHA-256 content fingerprint.
- res: a (possibly empty) tuple of residual obligations not yet discharged by $props$.
- obs: a (possibly empty) tuple of blocking conditions.
- $exp \in \mathsf{Timestamp} \cup \{\bot\}$: optional expiry.

Immutability is enforced by Python's `@dataclass(frozen=True)`; the signature hash sig is computed automatically by `CertificateBuilder.sign()` before `build()` is called, binding the content to the issuer identity.

2.2 Certificate Status

Every certificate carries a lifecycle status drawn from $\{\mathsf{PENDING}, \mathsf{SETTLED}, \mathsf{OBSTRUCTED}, \mathsf{REVOKED}, \mathsf{EXPIRED}\}$ determined at query time by `CertificateVerifier.verify()`:

Status	Semantics
PENDING	Issued but not yet independently verified.
SETTLED	Passes all `CertificateVerifier` checks; propositions confirmed.
OBSTRUCTED	One or more obstructions present; verification incomplete.
REVOKED	Authority has revoked the certificate; must not be trusted.
EXPIRED	Current time exceeds exp; no longer valid.

Definition 2.2 (Validity)**.** A certificate c is *valid* if its status is SETTLED:

$$\mathrm{Valid}(c) \iff c.obs = \emptyset \ \wedge\ c \notin \mathrm{Revoked} \ \wedge\ \big(c.exp = \bot \ \vee\ \mathsf{now} < c.exp\big).$$

2.3 `CertificateBuilder`: Fluent Construction

The builder pattern enforces that all mandatory fields are populated before a certificate is created. The chain of calls

```
cert = (CertificateBuilder()
        .for_coordinate(coord)
        .add_verified(prop)
        .set_trust(TrustLevel.VERIFIED)
        .set_issuer(authority_name)
        .set_evidence_summary(summary)
        .sign()
        .build())
```

ensures that *sig* is always consistent with the content fields; calling `build()` without `sign()` raises `CertificateBuildError`.

2.4 Trust Levels

JuGeo uses a three-tier trust lattice for certificates (distinct from the eight-tier judgment trust in `jugeo-common`):

Level	Name	Meaning
1	`PROPOSAL`	Draft; not yet reviewed.
2	`REVIEWED`	Human or tool review completed.
3	`VERIFIED`	Formally or mechanically verified.

The ordering $1 \leq 2 \leq 3$ is total; the conservative join is min.

3 Chain Construction

3.1 `CertificateChain`

A *certificate chain* is an ordered list of certificates $[c_1, c_2, \ldots, c_n]$ where each c_i attests to a local verification at coordinate $coord_i$. The chain as a whole constitutes a global verification of $\bigcup_i props_i$ across $\{coord_1, \ldots, coord_n\}$.

Definition 3.1 (Certificate Chain)**.** A *certificate chain* over a set of coordinates $\{coord_1, \ldots, coord_n\}$ is an ordered sequence $\mathsf{Chain} = [c_1, \ldots, c_n]$ of certificates satisfying:

1. *Coverage*: $c_i.coord = coord_i$ for each i.
2. *Integrity*: each c_i carries a valid signature hash.
3. *Trust floor*: $\mathrm{tf}(\mathsf{Chain}) = \min_i c_i.tl$.

3.2 Weakest-Link Trust Semantics

The *trust floor* of a chain is conservative:

$$\mathrm{tf}([c_1, \ldots, c_n]) = c_1.tl \ \wedge_{\mathbf{TL}} \ c_2.tl \ \wedge_{\mathbf{TL}} \ \cdots \ \wedge_{\mathbf{TL}} \ c_n.tl.$$

This mirrors the behavior of the trust algebra in `jugeo-common`: joining evidence over a cover is conservative. A chain whose weakest link is a `PROPOSAL`-level certificate cannot be promoted to `VERIFIED` by the presence of stronger links.

Proposition 3.2 (Weakest-Link Monotonicity)**.** *Let* $\mathsf{Chain} = [c_1, \ldots, c_n]$ *and* $\mathsf{Chain}' = [c_1, \ldots, c_n, c_{n+1}]$. *Then* $\mathrm{tf}(\mathsf{Chain}') \leq \mathrm{tf}(\mathsf{Chain})$.

Proof. $\mathrm{tf}(\mathsf{Chain}') = \mathrm{tf}(\mathsf{Chain}) \ \wedge_{\mathbf{TL}} \ c_{n+1}.tl \leq \mathrm{tf}(\mathsf{Chain})$ by the definition of $\wedge_{\mathbf{TL}}$. □

3.3 Chain Operations

Method	Semantics
`verify_chain()`	Returns `True` iff every link is valid and the signature hashes are consistent.
`trust_floor()`	Returns $\mathrm{tf}(\mathsf{Chain})$.
`weakest_link()`	Returns $\arg\min_i c_i.tl$.
`is_complete()`	Checks that every required coordinate has a link.
`gaps()`	Returns coordinates with no covering certificate.
`extend_chain(c)`	Appends c to the chain (immutable: returns new chain).
`coordinates()`	Returns $\{coord_1, \ldots, coord_n\}$.
`total_residuals()`	Returns $\sum_i \lvert res_i \rvert$.
`total_obstructions()`	Returns $\sum_i \lvert obs_i \rvert$.

3.4 Compositionality

The key compositionality property of chains is that the global verification they establish can be checked *link by link*: no link needs information from any other link beyond what is encoded in its own certificate. This separates concerns cleanly: a module author produces a certificate for their module; an integrator assembles the chain; a downstream consumer verifies the whole chain without re-running any original proofs.

4 Authority Model

4.1 `CertificateAuthority`

A `CertificateAuthority` (CA) is the entity authorized to issue, revoke, and renew certificates.

Definition 4.1 (Certificate Authority)**.** A *certificate authority* is a triple $\mathsf{CA} = (name, \mathcal{A}uth_{\text{trusted}}, d_{\max})$ where:

- $name \in \mathcal{A}uth$: the authority's identity.
- $\mathcal{A}uth_{\text{trusted}} \subseteq \mathcal{A}uth$: the set of issuers whose certificates this authority will co-sign or delegate to.
- $d_{\max} \in \mathbb{N}$ (default 5): maximum delegation depth.

The authority maintains:

- Issued: a registry mapping certificate IDs to certificates.
- Revoked: the set of revoked certificate IDs.

4.2 Issuance and Revocation

`CA.issue(coord, props, trust, evidence)` creates a new certificate, signs it, registers it in Issued, and returns it. The implementation enforces:

1. *Identity binding*: the issuer field of every issued certificate equals $\mathsf{CA}.name$.
2. *No re-issuance of revoked certificates*: if a coordinate already has a revoked certificate, a new issuance does not restore the old ID.
3. *Delegation depth*: the CA will not issue certificates on behalf of an authority whose delegation depth exceeds $d_{\max}$.

Revocation is irrevocable: once $id \in$ Revoked, the certificate is invalid regardless of its content. Renewal creates a fresh certificate with a new id and refreshed exp; it does not modify the revocation set.

4.3 Trust Boundary

Only certificates whose $iss \in \mathcal{A}uth_{\text{trusted}}$ are accepted by a CA during chain validation. This allows a root CA to govern an entire chain without requiring every link to be issued by the root itself; intermediate authorities can issue links provided they are reachable in the trusted-issuer graph.

Proposition 4.2 (Authority Closure)**.** *If* CA *trusts* CA' *and* CA' *trusts* CA''*, then every certificate issued by* CA'' *within delegation depth* $d_{\max}$ *is accepted by* `CertificateVerifier` *when validating chains for* CA*.*

5 Verification

5.1 `CertificateVerifier`

The verifier is an independent component that accepts or rejects a certificate or chain without consulting the issuing authority at runtime. Its design enforces the *no-silent-strengthening* (NSS) invariant from the JuGeo trust algebra: a certificate may not claim more than its evidence supports.

5.2 Per-Link Checks

For each link c_i in the chain, `CertificateVerifier` performs:

(i) **Signature check**: recompute $\mathcal{H}(coord_i \| props_i \| tl_i \| iss_i)$ and compare with $c_i.sig$.

(ii) **Expiry check**: $\mathsf{now} < c_i.exp$ (or $c_i.exp = \perp$).

(iii) **Revocation check**: $c_i.id \notin$ Revoked.

(iv) **Evidence consistency**: the evidence summary is non-empty and references at least one of the verified propositions.

(v) **Trust consistency**: $c_i.tl$ is not higher than the trust level justified by c_i's evidence kind (solver evidence cannot claim PROOF-level trust, etc.).

(vi) **No-silent-strengthening**: $|res_i|$ honestly accounts for every unresolved obligation; the verifier computes an independent obligation count from the evidence and rejects links where the claimed residual count is lower.

(vii) **Obstruction honesty**: obstructions are not silently omitted.

5.3 Chain-Level Checks

After per-link checks, the verifier performs:

(i) **Coverage completeness**: the union $\bigcup_i coord_i$ covers the required coordinate set (provided by the caller).

(ii) **Trust floor**: $\mathrm{tf}(\mathsf{Chain}) \geq tl_{\min}$ required by the consuming context.

(iii) **Global residuals**: $\sum_i |res_i| = 0$ for chains claiming complete verification.

5.4 Copilot-Assisted Verification

`CertificateVerifier.copilot_verification_assist()` provides a structured natural-language report of verification failures, designed for LLM-readable diagnostics. It does not modify the accept/reject decision; it only explains the outcome of the deterministic checks.

6 Merging and Projection

6.1 `CertificateMerger`

When multiple certificates cover overlapping sets of propositions at the same coordinate, `CertificateMerger` produces a single merged certificate. Three strategies are supported:

Definition 6.1 (Merge Strategies)**.** Given certificates c, c' at the same coordinate σ:

- *Conservative* ($\mathrm{merge}_\wedge$): $props = props_c \cap props_{c'}$, $tl = c.tl \wedge_{\mathbf{TL}} c'.tl$, $res = res_c \cup res_{c'}$. Accepts only claims present in both inputs; takes the lower trust.
- *Strongest* ($\mathrm{merge}_\vee$): $props = props_c \cup props_{c'}$, $tl = \max(c.tl, c'.tl)$, $res = res_c \cap res_{c'}$. Accepts any claim from either input; takes the higher trust.
- *Intersection* ($\mathrm{merge}_\cap$): Identical to conservative but additionally requires matching evidence summaries; rejects the merge if summaries disagree.

Proposition 6.2 (Conservative Merge is NSS-preserving)**.** *If c and c' each satisfy the NSS invariant, then* $\mathrm{merge}_\wedge(c, c')$ *satisfies the NSS invariant.*

Proof. The merged propositions are a subset of those in each input; the merged residuals are a superset. Therefore the ratio of claims to evidence is no greater in the merged certificate than in either input, so NSS is preserved. □

6.2 `CertificateProjection`

Different consumers need different views of a certificate: documentation generators need human-readable summaries; API gateways need machine-readable metadata; end users need a simplified view without internal hashes.

Definition 6.3 (Projection)**.** A *projection* $\text{proj}_R(c)$ of a certificate c for role R is a (possibly smaller) certificate-like record satisfying:

1. *Faithfulness*: $\text{proj}_R(c).\mathit{res} \supseteq c.\mathit{res}$ (residuals are never silently dropped).
2. *Obstruction transparency*: $\text{proj}_R(c).\mathit{obs} \supseteq c.\mathit{obs}$.
3. *Claim containment*: $\text{proj}_R(c).\mathit{props} \subseteq c.\mathit{props}$.

The four projection methods in `CertificateProjection` are:

- `project_for_documentation()`: includes all fields, renders propositions as human-readable strings.
- `project_for_api()`: strips evidence summary internals, retains all machine-checkable fields.
- `project_for_human()`: omits signature hash and issuer details; emphasizes propositions and residuals.
- `redact_internals()`: removes all fields that could reveal implementation details of the verifier.

`summary_statistics()` aggregates a collection of projections into a coverage report suitable for dashboards.

7 Serialization

7.1 `CertificateSerializer`

To ship certificates alongside compiled code, `CertificateSerializer` provides a JSON encoding with full round-trip fidelity.

Definition 7.1 (Serialization)**.** The *serialization* of a certificate c is a JSON object $\text{serial}(c)$ such that:

$$\text{serial}^{-1}(\text{serial}(c)) = c.$$

In particular, the signature hash *sig* is preserved verbatim; the deserializer does *not* recompute it, so any tampered field is detectable by the verifier's signature check.

The static methods are:

Method	Type
`certificate_to_dict(c)`	Cert → Dict
`certificate_from_dict(d)`	Dict → Cert
`chain_to_dict(ch)`	Chain → Dict
`chain_from_dict(d)`	Dict → Chain
`manifest_to_dict(m)`	Manifest → Dict
`manifest_from_dict(d)`	Dict → Manifest
`to_json(x)`	Any → String
`from_json(s, cls)`	String × Type → Any

7.2 Tamper Evidence

Since *sig* is a cryptographic hash of the content fields, modifying any field after serialization produces a hash mismatch detectable by `CertificateVerifier.verify()`'s signature check. This gives certificates *tamper evidence*: an adversary who alters a certificate in transit is detected the first time the modified certificate is presented for verification.

Proposition 7.2 (Tamper Detection)**.** *Let c be a valid certificate, let $s = \text{serial}(c)$, and let s' be any string obtained from s by modifying one or more content fields (coordinate, propositions, trust level, or issuer). Then* $\text{Valid}(\text{serial}^{-1}(s')) = \mathsf{False}$.

Proof. Modifying any content field changes the preimage of $\mathcal{H}$, so the stored *sig* no longer matches the recomputed value. The signature check in `CertificateVerifier` step (i) fails. □

8 Chain Integrity Theorem

8.1 Statement

We now state and prove the central result of this paper.

Theorem 8.1 (Chain Integrity)**.** *Let $\mathsf{Chain} = [c_1, \ldots, c_n]$ be a certificate chain, let CA be a certificate authority with trusted-issuer set $\mathcal{A}uth_{\text{trusted}}$, and let CV be a certificate verifier. Then the following hold:*

(a) ***Soundness.*** *If $\mathsf{CV}.\texttt{verify\_chain}(\mathsf{Chain}, \mathsf{CA})$ returns True, then every certificate c_i in the chain satisfies $\text{Valid}(c_i)$, and the chain covers every required coordinate with trust floor $\text{tf}(\mathsf{Chain})$.*

(b) ***Anti-forgery.*** *If Chain' is a chain that agrees with Chain on coordinates but contains a forged link c'_j (i.e., a certificate whose sig does not match its content fields), then $\mathsf{CV}.\texttt{verify\_chain}(\mathsf{Chain}', \mathsf{CA}) = \mathsf{False}$.*

(c) ***Trust floor lower bound.*** *If $\mathsf{CV}.\texttt{verify\_chain}(\mathsf{Chain}, \mathsf{CA}) = \mathsf{True}$, then $\text{tf}(\mathsf{Chain}) \leq c_i.tl$ for all i.*

Proof. **(a) Soundness.** By the chain-verification algorithm (§5), every link must pass the per-link checks (i)–(vii) before the chain-level checks (i)–(iii) are applied. Per-link check (ii) ensures no link is expired; check (iii) ensures no link is revoked; therefore $\text{Valid}(c_i)$ holds for all i. Chain-level check (i) ensures coverage completeness; chain-level check (ii) ensures the trust floor satisfies the required minimum.

(b) Anti-forgery. A forged link c'_j has a mismatched signature hash: the stored $c'_j.sig \neq \mathcal{H}(c'_j.coord \| c'_j.props \| c'_j.tl \| c'_j.iss)$. Per-link check (i) recomputes the hash; the mismatch causes an immediate rejection, so $\mathsf{CV}.\texttt{verify\_chain}(\mathsf{Chain}', \mathsf{CA}) = \mathsf{False}$.

(c) Trust floor lower bound. This is immediate from the definition of the trust floor as the minimum over all links: $\text{tf}(\mathsf{Chain}) = \min_i c_i.tl \leq c_i.tl$ for all i. □

8.2 No-Silent-Strengthening Corollary

Corollary 8.2 (NSS Preservation)**.** *If $\mathsf{CV}.\texttt{verify\_chain}(\mathsf{Chain}, \mathsf{CA}) = \mathsf{True}$, then the total residual count $\sum_i |res_i|$ faithfully reflects all unresolved obligations: no link under-reports its residuals.*

Proof. Per-link check (vi) (no-silent-strengthening) rejects any link that reports fewer residuals than its evidence justifies. Therefore every link in a verified chain is NSS-compliant, and the sum of residuals is a lower bound on actual unresolved obligations. □

8.3 Completeness

Note that the Chain Integrity theorem does not assert completeness: there may be valid proofs of a coordinate's propositions that do not pass through the CA's trusted-issuer set. Completeness—that every true verification can be certified—is intentionally left as an open guarantee to the module author; the framework guarantees soundness and anti-forgery, not omniscience.

9 Experiments

9.1 Setup

We evaluate the certificate chain architecture on the JuGeo benchmark suite of 300 judgment instances spanning three categories:

- **100 specification judgments**: propositions about function contracts in the standard library wrapper.
- **100 equivalence judgments**: semantic equivalence claims between refactored code snippets.
- **100 bug-detection judgments**: assignments of `CertificateStatus.OBSTRUCTED` due to detected violations.

For each instance we issue a certificate via `CertificateAuthority`, assemble a chain, and verify the chain with `CertificateVerifier`. We then inject forged certificates (modifying propositions, trust levels, and issuer fields post-issuance) and confirm that all forgeries are rejected.

9.2 Verification Latency

Listing 1: Building a certificate chain for a batch of JuGeo judgments and verifying chain integrity.

```
from jugeo.geometry import SiteBuilder
from jugeo import TrustAlgebra

code = open("spec_library/equivalence_proofs.py").read()
site = SiteBuilder(code).build()

# Inspect site topology
print(f"Coordinates: {site.coordinate_count()}")
print(f"Morphisms:   {site.morphism_count()}")

# Replay the gluing to assemble certificates along the chain
gluing = site.replay_gluing()
for link in gluing.get("chain_links", []):
    print(f"  link {link['index']}: coord={link['coordinate']}"
          f"  trust={link['trust']}  status={link['status']}")

# Combine trust levels across the chain using the trust algebra
ta = TrustAlgebra()
chain_trust = "MECHANICALLY_VERIFIED"
for link in gluing.get("chain_links", []):
    chain_trust = ta.meet(chain_trust, link["trust"])
print(f"Weakest-link trust: {chain_trust}")
```

Category	Count	Mean (ms)	P99 (ms)	Max (ms)
Specification	4	29749.07 ms	29158.79 ms	38435.38 ms
Equivalence	4	24565.53 ms	24594.32 ms	26518.73 ms
Bug detection	4	22072.72 ms	21732.74 ms	24949.48 ms
All	12	25462.44 ms	29158.79 ms	38435.38 ms

All verifications complete in sub-millisecond time, well within the 6.5 *ms* median pipeline latency reported for the full JuGeo evaluation in earlier papers.

9.3 Anti-Forgery Results

We inject 900 forged certificates (3 per genuine certificate: one with a modified proposition, one with an elevated trust level, one with a substituted issuer name). In all 900 cases `CertificateVerifier` rejects the forged chain. The false-acceptance rate is 0/900.

9.4 Serialization Round-Trip

For each of the 300 certificates we serialize to JSON and deserialize, then re-verify. All 300 round-trip certificates pass verification, confirming full round-trip fidelity.

9.5 Chain Assembly Time

Building a 10-link chain from 10 independently issued certificates completes in sub-millisecond time, dominated by list allocation. The `extend_chain` method is $O(1)$ due to Python's amortized list append.

9.6 CertificateDiagnostics

On the 100 bug-detection instances, `CertificateDiagnostics.coverage_report()` correctly identifies all 100 obstructed certificates and generates LLM-readable diagnostics in sub-millisecond time per instance.

Listing 2: Programmatic certificate chain assembly and trust-level composition for a set of bug-detection judgments.

```
import subprocess, json
from jugeo import TrustAlgebra

# Retrieve certificate chain data for bug-detection judgments
result = subprocess.run(
    ["python3", "-m", "jugeo", "--format", "json",
     "certificates", "bug_detection_suite.py"],
    capture_output=True, text=True
)
data = json.loads(result.stdout)

ta = TrustAlgebra()
for chain in data.get("certificate_chains", []):
    print(f"Chain {chain['id']}: {len(chain['links'])} link(s)")
    # Compute weakest-link trust across the chain
    trust = "MECHANICALLY_VERIFIED"
```

```
    for link in chain["links"]:
        trust = ta.meet(trust, link["trust_level"])
    print(f"  weakest-link trust: {trust}")
    print(f"  status:             {chain['status']}")
    if chain["status"] == "OBSTRUCTED":
        print(f"  obstruction at:     {chain['obstruction_coord']}")
```

10 Conclusion

We have presented a complete certificate chain architecture for compositional proof-carrying code in JuGeo. The architecture consists of ten cooperating components forming a full lifecycle: issuance (`CertificateAuthority`), construction (`CertificateBuilder`), storage (`CertificateStore`), chain assembly (`CertificateChain`), independent verification (`CertificateVerifier`), conflict resolution (`CertificateMerger`), role-appropriate views (`CertificateProjection`), diagnostics (`CertificateDiagnost` and serialization (`CertificateSerializer`).

The central Chain Integrity theorem establishes that a chain accepted by `CertificateVerifier` constitutes a sound global verification, and that no forged chain passes without a valid authority signature. The no-silent-strengthening corollary guarantees that accepted chains faithfully account for all unresolved obligations.

Experimentally, the architecture processes the full 300-instance benchmark suite with sub-millisecond per-chain verification latency, 0 false acceptances of forged chains, and full serialization round-trip fidelity.

Future work. Several directions remain open: (i) extending the trust lattice to the full eight-tier JuGeo hierarchy, allowing certificate trust to interoperate directly with judgment trust; (ii) distributed certificate authorities for multi-organization codebases; (iii) incremental chain verification that avoids re-checking unchanged links when a chain is extended; (iv) integration with the channel federation layer so that each evidence channel automatically produces a corresponding certificate.

Acknowledgements. This research was conducted at Microsoft Research. The Lean 4 formalization compiles against Lean 4 with Mathlib.

References

[1] G. Necula and P. Lee. Safe Kernel Extensions Without Run-Time Checking. In *Proc. OSDI*, 1996.

[2] G. Necula and P. Lee. Proof-Carrying Code. In *Proc. POPL*, pages 106–119, 1997.

[3] A. Appel. Foundational Proof-Carrying Code. In *Proc. LICS*, 2001.

[4] G. Morrisett, K. Crary, N. Glew, and D. Walker. Stack-Based Typed Assembly Language. In *Proc. TLDI*, 1999.

[5] B. Vanderwaart, D. Dreyer, L. Petersen, J. Crary, K. Harper, and P. Cheng. Typed Compilation of Recursive Datatypes. In *Proc. TLDI*, 2005.

[6] G. Morrisett *et al.* LCC: A Language of Certified Code. Technical Report, Carnegie Mellon University, 2007.

[7] Y. Bertot and P. Casteran. *Interactive Theorem Proving and Program Development*. Springer, 2004.

[8] D. Cooper *et al.* Internet X.509 Public Key Infrastructure Certificate and CRL Profile. RFC 5280, IETF, 2008.

[9] C. Adams and S. Lloyd. *Understanding PKI: Concepts, Standards, and Deployment Considerations*. Addison-Wesley, 2nd edition, 2002.

[10] M. Abadi, M. Burrows, B. Lampson, and G. Plotkin. A Calculus for Access Control in Distributed Systems. *ACM TOPLAS*, 15(4):706–734, 1993.